

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



CRUD Completo na API REST

Validação, Tratamento de Exceções, Respostas HTTP
Introdução ao S.O.L.I.D. com a Responsabilidade Única

Prof. Gill Velleda Gonzales

gillgonzales@ifsul.edu.br | [@gillgonzales](https://www.instagram.com/gillgonzales) | [@g1ll](https://www.instagram.com/g1ll)

CSTSI - Novembro - 2024

CRUD Completo na API REST



Tópicos da Aula

- **Criação do Método Store**
 - Exemplo básico de registro de dados na API
 - Resposta JSON simplificada
- **Validação e Regras de Validação**
 - Validação de dados da requisição
 - Tipos de Regras de Validação
- **Tratamento de Erros**
 - Captura e Lançamento de Exceções
 - Tipos de Exceções (*instanceOf*, *catchs*)
- **Refatoração e Princípio da Responsabilidade Única (SOLID)**
 - Criação da classe **Form Request** para **Validação**
 - Criação da classe **Resource Json** para o método **Store**
 - **Controle de Exceção** e Criação de **Exceção Própria**
 - Métodos **Update** e **Delete**
 - Reutilização de **código refatorado** e princípio **SOLID**.
- **Atividades.**

Método Store (o CREATE do Crud)

Implementação do método store no controller de Produto da API REST



- O método **store()** será semelhante ao método criado para a aplicação web.
- No entanto, alteramos inicialmente o retorno, como um objeto JSON.

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store

```
15  class ProdutoController extends Controller
27
28      public function store(Request $request)
29      {
30          $produto = $request->all();
31          $produto['importado'] = $request->has('importado');
32
33          if(Produto::create($produto)){
34              return response()->json('Produto Criado!', 201);
35          }else {
36              return response()->json('Erro ao criar produto!', 500);
37          }
38      }
```

Método Store (o CREATE do Crud)

Implementação do método store no controller de Produto da API REST



- Retornamos um JSON com a resposta criada pelo método **json()** do objeto de resposta retornado pela função **response()**, um **helper**.
- Alteramos a mensagem e o status http de acordo com o retorno da operação **create()** da Model de Produto.
- Esta é um versão muito simplificada, vários erros podem ocorrer neste contexto de código, como por exemplo, o **NÃO ENVIO** de campos **obrigatórios (required)**.

```
15  class ProdutoController extends Controller
28      public function store(Request $request)
32
33          if(Produto::create($produto)){
34              return response()->json('Produto Criado!', 201);
35          }else {
36              return response()->json('Erro ao criar produto!', 500);
37          }
38      }
```


Método Store (o CREATE do Crud)

Implementação do método store no controller de Produto da API REST



- Vamos adicionar então uma validação ao código anterior e verificar se o erro é de validação, da classe **400 (Bad request)** ou realmente um erro interno do servidor (**500**).
- Portanto usamos a função **validate** do objeto de requisição para validar os campos de entrada, inicialmente apenas o campo de **preço**, com no exemplo:

```
app > Http > Controllers > Api >  ProdutoController.php > PHP Intelephense >  ProdutoController >  store  
15  class ProdutoController extends Controller  
28      public function store(Request $request)  
29      {  
30          $request->validate([  
31              "preco"=>"required"  
32          ]);
```

- O método **validate()** recebe um *array* com as regras de validação (**rules**).
- No exemplo acima, o campo **preço** está definido como **obrigatório (required)**

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- **Método Validate**

- A validação dos dados de entrada é feita a partir do objeto de requisição, ou seja, o objeto ***Request***;
- O método ***validate*** recebe como parâmetro do tipo *array* associativo contendo os campos que devem ser validados.
- Cada campo a ser validado recebe uma *string* ou *array* que defini as **regras** de validação, ***validation rules***.
- Se as entradas forem inválidas o fluxo da aplicação é interrompido, sendo gerada uma exceção, que deverá retornar como **código http** de resposta o status **422**.

Exemplo: `$request→validate([`
 `“email”      ⇒ “required | email”,`
 `“password”  ⇒ “required | min:8”,`
 `])`

- Para alterar a resposta padrão de erro será necessário usar um bloco ***try/catch*** e capturar a **exceção** quando as entradas forem inválidas.

Validação da requisição



- **Método Validate**

- As regras também podem ser definidas como arrays, embora não seja o mais usual:

Exemplo: `$request` → `validate([`
 `“email”` ⇒ `[“required”, “email”],`
 `“password”` ⇒ `[“required”, “min:8”],`
 `])`

- Novamente, se a validação falhar, o Laravel retornará a resposta apropriada, que é uma página de erro, caso a nossa aplicação não trate a **exceção** gerada (**try/catch**).
- Nos nossos exemplos, iremos sempre capturar a exceção e retornar uma resposta em formato **JSON** para a **API REST**.
- Se a Validação passar, ou seja, os dados estão de acordo com as **Regras** definidas, então o fluxo da aplicação seguirá normalmente.

- Nos nossos exemplos, iremos sempre capturar a exceção e retornar uma resposta em formato **JSON** para a **API REST**.

- Se a Validação passar, ou seja, os dados estão de acordo com as **Regras** definidas, então o fluxo da aplicação seguirá normalmente.

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- Versão com um bloco **try/catch** para capturar a **exceção** gerada.

```
app > Http > Controllers > Api >  ProdutoController.php > PHP Intelephense >  ProdutoController >  store
15  class ProdutoController extends Controller
28      public function store(Request $request)
29  {
30      try {
31          $request->validate(["preco" => "required"]);
32          $produto = $request->all();
33          $produto['importado'] = $request->has('importado');
34          Produto::create($produto);
35          return response()->json('Produto Criado!', 201);
36      } catch (Exception $error) {
37          $httpStatus = 500;
38          if($error instanceof ValidationException) $httpStatus=422;
39          return response()->json($error->getMessage(), $httpStatus);
40      }
41  }
```

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- Podemos identificar se é uma exceção de validação verificando se o objeto é uma instância de **ValidationException** através do operador **instanceOf**.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelphense > ProdutoController > store

15  class ProdutoController extends Controller
28      public function store(Request $request)
29      {
30          try {
31              $request->validate(["preco" => "required"]);
32              $produto = $request->all();
33              $produto['importado'] = $request->has('importado');
34              Produto::create($produto);
35              return response()->json('Produto Criado!', 201);
36          } catch (Exception $error) {
37              $httpStatus = 500;
38              if($error instanceof ValidationException) $httpStatus=422;
39              return response()->json($error->getMessage(), $httpStatus);
40          }
41      }
```

IFSUL | CSTSI | Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br | @gillgonzales | @g1ll

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- **Regras de Validação** - Existem diversas regras de validação, abaixo as mais comuns:

- **unique**: Verifica se o valor é único no banco de dados, ou seja, não deverá existir um valor já cadastrado.

Exemplo: `unique:table,column`

`"email"` $\Rightarrow$ `"unique:users"`

`"email"` $\Rightarrow$ `"unique:users, email_address"`

OBS: Caso a coluna da tabela não seja igual ao nome do campo, deve se especificar após a virgula.

- **required**: Mais básica, define o campo como obrigatório. Esta regra indica que o parâmetro não pode faltar e seu conteúdo não pode ser uma string vazia.

Exemplo: `"email"` $\Rightarrow$ `"required"`

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- **Regras de Validação:**

- **nullable:** Avisamos ao validador que esta opção pode ser nula e não deve invalidar a requisição caso não exista. Normalmente utilizada em combinação com outra regra.

Exemplo:

“importado” ⇒ “nullable | boolean”

- **boolean:** Determina que o parâmetro deve ser um booleano. Aceita os seguintes valores: *true*, *false*, *1*, *0*, “1”, and “0”.
- **min, max:** Determina o valor mínimo e máximo, possui comportamentos diferentes dependendo do tipo de dado, suporta: numeros, strings, arrays e arquivos.

Exemplo: “qtd_estoque” ⇒ “required | min:1”

“descricao” ⇒ “required | max:500”

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- **Regras de Validação:**

- **size:** Determina o tamanho exato do conteúdo do parâmetro, depende do tipo da variável, para strings é a quantidade de caracteres, para arrays o quantidade de itens, para números o valor inteiro ou numérico, para arquivos, o tamanho em bytes.

Exemplo: “tags” => “size:5”

- **mimes:** Determina o tipo do arquivo, ou extensão.

Exemplo: 'photo' => 'mimes:jpg,bmp,png'

- **current_password (v9):** A senha informada deverá ser igual à senha do usuário autenticado. Neste caso deverá ser especificado um **guard** middleware de autenticação.

- Exemplo: 'password' => 'current_password:api'

- **Documentação completa das Regras suportadas!**

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- Regras de validação completas para os campos da model de Produto.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelphense > ProdutoController > store
15  class ProdutoController extends Controller
28      public function store(Request $request)
29      {
30          try {
31              $request->validate([
32                  "nome" => "required | max: 10",
33                  "descricao" => "required | max:500",
34                  "qtd_estoque" => "required | numeric | min:2",
35                  "preco" => "required | numeric | min: 1.99",
36                  "importado" => "nullable | boolean",
37              ]);
38              $produto = $request->all();
39              $produto['importado'] = $request->has('importado');
40              Produto::create($produto);
```

Validação da requisição

Validando dados da requisição para métodos POST, PUT ou PATCH



- **Testando a Validação com o Postman:**

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store

```
15 class ProdutoController extends Controller
28     public function store(Request $request)
42     } catch (Exception $error) {
43         $httpStatus = 500;
44         if($error instanceof ValidationException) $httpStatus=422;
45         return response()->json(["erro"=>$error->getMessage()], $httpStatus);
46     }
47 }
```

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

☐	preco	Text	2
--------------------------	-------	------	---

Body Cookies Headers (7) Test Results Status: 422 Unprocessable Content Time: 46 ms Size: 298 B Save as example ...

Pretty Raw Preview Visualize JSON

```
1 {
2   "erro": "The preco field is required. (and 1 more error)"
3 }
```

Gerenciamento de Exceção

Lidando com erros da aplicação



- **Relançando** a exceção do tipo **ValidationException**:
 - Para não perder o formato padrão de mensagem de erros da validação, relançamos como **TRHOW** a exceção quando for do tipo **ValidationException**.
 - O próprio *framework* deverá gerar a respostas com o status adequado, **422 (Unprocessable Content)**, conteúdo não processável.
 - Poderemos visualizar então todos os campos com erros, mantendo a mensagem padrão do **Laravel**.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store
15  class ProdutoController extends Controller
28      public function store(Request $request)
42      {
43          if($error instanceof ValidationException) throw $error;
44          return response()->json(["Erro"=>$error->getMessage()],500);
45      }
46  }
```

IFSUL | CSTSI | Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br | @gillgonzales | @g1ll

Gerenciamento de Exceção

Lidando com erros da aplicação



POST ▼ http://localhost:8085/api/produtos Send ▼

Params Authorization Headers (10) **Body** ● Scripts Settings Cookies

☐ none ☒ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

☐	preco	Text ▼	2
☒	importado	Text ▼	"on"

Body Cookies Headers (7) Test Results 🌐 Status: 422 Unprocessable Content Time: 48 ms Size: 412 B 💾 Save as example ⋮

Pretty Raw Preview Visualize JSON ▼ ≡

```
1 {
2   "message": "The preco field is required. (and 1 more error)",
3   "errors": {
4     "preco": [
5       "The preco field is required."
6     ],
7     "importado": [
8       "The importado field must be true or false."
9     ]
10  }
11 }
```

O campo importado deve ser 0 ou 1 para ser considerado false ou true

Gerenciamento de Exceção

Lidando com erros da aplicação



- **Relançando** a exceção do tipo **ValidationException**:
 - Para não perder o formato padrão de mensagem de erros da validação, relançamos como **TRHOW** a exceção quando for do tipo **ValidationException**.
 - O próprio *framework* deverá gerar a respostas com o status adequado, **422 (Unprocessable Content)**, conteúdo não processável.
 - Poderemos visualizar então todos os campos com erros, mantendo a mensagem padrão do **Laravel**.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store
15  class ProdutoController extends Controller
28      public function store(Request $request)
42      {
43          if($error instanceof ValidationException) throw $error;
44          return response()->json(["Erro"=>$error->getMessage()],500);
45      }
46  }
```

IFSUL | CSTSI | Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br | @gillgonzales | @g1ll

Gerenciamento de Exceção

Lidando com erros da aplicação



- **Relançando** a exceção do tipo **ValidationException**:
 - Uma outra forma de detectar uma exceção específica, como a **ValidationException**, seria usar um **catch** adicional e intermediário.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store
16  class ProdutoController extends Controller
29      public function store(Request $request)
46      {
47          // ...
48          } catch (ValidationException $error) {
49              throw $error;
          } catch (Exception $error) {
              // if ($error instanceof ValidationException) throw $error;
          }
      }
```

- Neste caso apenas o primeiro bloco **catch** será executado quando ocorrer uma exceção do tipo **ValidationException**.
- Pois como relançamos com o **throw**, a exceção será *passada para frente* na pilha de chamadas de classes da aplicação.

Gerenciamento de Exceção

Lidando com erros da aplicação



- Da maneira como estamos retornando o erro, não temos muitos detalhes da exceção gerada, como por exemplo, em que arquivo ocorreu ou quais classes foram executadas no **fluxo da aplicação** até gerar o erro, ou seja, o **stack trace**.

app > Http > Controllers > Api > ProdutoController.php > PHPintelephense > ProdutoController > store

```
16 class ProdutoController extends Controller
29     public function store(Request $request)
46     {
47         } catch ( ValidationException $error) {
48             throw $error;
49         } catch (Exception $error) {
50             // if ($error instanceof ValidationException) throw $error;
            return response()->json(["Erro"=>$error->getMessage()], 500);
```

Body Cookies Headers (7) Test Results

Status: 500 Internal Server Error Time: 29 ms Size: 530 B Save as example

Pretty

Raw

Preview

Visualize

JSON



```
1 {
2   "Erro": "SQLSTATE[HY000] [2002] Connection refused (Connection: mysql, SQL: insert into `produtos` (`nome`,
   `descricao`, `qtd_estoque`, `preco`, `importado`, `updated_at`, `created_at`) values (Solid test, Testando
   insercao produto Api, 2, 2, 0, 2024-11-10 15:38:12, 2024-11-10 15:38:12))"
3 }
```

Gerenciamento de Exceção

Lidando com erros da aplicação



- Alteramos o nosso segundo **catch** para fornecer mais detalhes do erro gerado na exceção genérica, quando não for de validação, considerando a variável de ambiente **APP_DEBUG**.

```
48  } catch (Exception $error) {
49      $httpStatus = 500;
50      $message = "Não foi possível criar novo produto!";
51      $response = ["Erro:" => $message];
52      if (env('APP_DEBUG'))
53      {
54          $response = [
55              ...$response,
56              'message' => $error->getMessage(),
57              'exception' => $error,
58              'trace' => $error->getTrace(),
59          ];
60      }
    return response()->json($response, $httpStatus);
}
```

Retornando um Resource no Método Store



Retorno do ProdutoResource no cadastro com sucesso.

- Até o momento estamos retornando uma resposta JSON simples, mas podemos retornar o **ResourceProduto**, também no método **Store**.
- Para adicionar ao recurso a mensagem e sucesso, usamos o método **additional**, após instanciar o recurso, e envolto por parenteses (**new Resource**), linha 42.
- Chamamos também o método **response** para retornar um objeto **ResponseJson** e depois o método **setStatusCode**, para alterar o Status HTTP

```
16  class ProdutoController extends Controller
29      public function store(Request $request)
30      {
39          $produto = $request->all();
40          $produto['importado'] = $request->has('importado');
41          $novoProduto = Produto::create($produto);
42          return (new ProdutoResource($novoProduto))
43              ->additional(["message" => 'Produto criado com sucesso!'])
44              ->response()->setStatusCode(201, "Produto Criado!");
```

Método Store

Pronto, mas saturado de responsabilidades.

- Após lidar com a validação, definindo as regras para cada campo de entrada, bem como os possíveis erros, através de exceções genéricas (**Exception**) ou específicas, como a **ValidationException**, e finalmente alterar o tipo de retorno para um recurso, **ResourceJson**, modificando o status HTTP, nosso método está pronto, mas **saturado de responsabilidades**.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store
16  class ProdutoController extends Controller
29      public function store(Request $request)
30      {
31          try {
32              $request->validate([
33                  "nome" => "required | max:20",
34                  "descricao" => "required | max:300",
35                  "qtd_estoque" => "required | numeric | min:1",
36                  "preco" => "required | numeric | min:1.99",
37                  "importado" => "nullable | boolean",
38              ]);
39              $produto = $request->all();
40              $produto['importado'] = $request->has('importado');
41              $novoProduto = Produto::create($produto);
42              return (new ProdutoResource($novoProduto))
43                  ->additional(["message" => 'Produto criado com sucesso!'])
44                  ->response()
45                  ->setStatusCode(201, "Produto Criado!");
46          } catch ( ValidationException $error) {
47              throw $error;
48          } catch (Exception $error) {
49              $message = "Não foi possível criar novo produto!";
50              $response = ["Erro:" => $message];
51              if (env('APP_DEBUG'))
52                  $response = [
53                      ...$response,
54                      'message' => $error->getMessage(),
55                      'exception' => $error,
56                      'trace' => $error->getTrace(),
57                  ];
58              return response()->json($response, 500);
59          }
60      }
```


Método Store

Pronto, mas saturado de responsabilidades.

- O controlador não deve acumular funções, mas apenas delegar, ser o intermediário, da **requisição** até a **resposta**.
- **Requisição:** **Validação** e **regras** na classe FormRequests, para cada tipo de validação, também **middlewares**.
- **Resposta:** classes **ResourceJson** específicas e **middlewares**.
- **Erros:** criar método único e/ou uma exceção especial para lidar com tipos de erros genéricos (**Exception**) ou específicos, (**ValidationException**).

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store
16  class ProdutoController extends Controller
29      public function store([Request $request])
30      {
31          try {
32              $request->validate([
33                  "nome" => "required | max:20",
34                  "descricao" => "required | max:300",
35                  "qtd_estoque" => "required | numeric | min:1",
36                  "preco" => "required | numeric | min:1.99",
37                  "importado" => "nullable | boolean",
38              ]);
39              $produto = $request->all();
40              $produto['importado'] = $request->has('importado');
41              $novoProduto = Produto::create($produto);
42              return (new ProdutoResource($novoProduto))
43                  ->additional(["message" => 'Produto criado com sucesso!'])
44                  ->response()
45                  ->setStatusCode(201, "Produto Criado!");
46          } catch ( ValidationException $error) {
47              throw $error;
48          } catch (Exception $error) {
49              $message = "Não foi possível criar novo produto!";
50              $response = ["Erro:" => $message];
51              if (env('APP_DEBUG'))
52                  $response = [
53                      ...$response,
54                      'message' => $error->getMessage(),
55                      'exception' => $error,
56                      'trace' => $error->getTrace(),
57                  ];
58              return response()->json($response, 500);
59          }
60      }
61  }
```


REFATORANDO: Form Requests para Validação

Criação e utilização da classe Form Requests para Validações



- Form Requests:

- O uso da classe Form Request é uma maneira mais simples de criar a validação para o recurso que necessitamos e separar esta validação do controlador, podendo ser personalizada para diferentes métodos como ***Store*** e ***Update***.
- Form Request também é indicado para cenários de validações complexas ou de autenticação;
- Para criar um ***Form Request*** de **Produtos** vamos executar o seguinte comando no artisan:

```
$> php artisan make:request ProdutoStoreRequest
```

Form Request

Configurando a validação com FormRequests.

- Será criada a classe ***ProdutosStoreRequest*** no diretório ***Http/Requests***.
- As regras de validação são configuradas sobrescrevendo o método ***rules()*** da classe pai ***FormRequest***.
- Devemos também alterar o método ***authorize*** para retornar ***true***, caso não necessite de algum tipo de verificação de usuário ou autenticação.
- Já aquela verificação do campo importado, é realizada pelo método ***prepareForValidation***, que ocorre antes da validação.
- **Documentação**

```
app > Http > Requests > ProdutoStoreRequest.php > PHP Intelephense > ProdutoStoreRequest > rules
7  class ProdutoStoreRequest extends FormRequest
12     public function authorize(): bool
13     {
14         return true;
15     }
16
17     /**
18      * Get the validation rules that apply to the request
19      *
20      * @return array<string, \Illuminate\Contracts\Validation
21      */
22     public function rules(): array
23     {
24         return [
25             "nome"=>"required | max:20",
26             "descricao"=>"required | max:300",
27             "qtd_estoque"=>"required | numeric | min:1",
28             "preco"=>"required | numeric | min:1.99",
29             "importado"=>"nullable | boolean",
30         ];
31     }
32
33     protected function prepareForValidation(): void
34     {
35         $this->merge([
36             'importado'=>$this->has('importado')
37         ]);
38     }
39 }
```

Form Request

Utilizando a nova classe no método store do ProdutoController.



- Para utilizar a validação configurada na classe **ProdutoStoreRequest**, devemos “**tipar**” a requisição recebida pelo método com esta nova classe.
- Desta forma, antes mesmo do nosso método ser executado, os **middlewares** que tratam o ciclo de requisição no *framework*, farão a validação e também o redirecionamento para a resposta de erro, caso ocorra algum.

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store > \$request

```
16 class ProdutoController extends Controller
63     public function store(ProdutoStoreRequest $request)
64     {
65         try {
```

Form Request

Utilizando a nova classe no método store do ProdutoController.



- Para repassar ao model de Produto os parâmetros válidos, já recebidos pelo controlador através do objeto \$request, utilizamos o método **validated()**.
- Será retornado pelo **validated()** um *array* com os campos **válidos** a serem inseridos no novo registro de produto.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelphense > ProdutoController > store
16  class ProdutoController extends Controller
29      public function store(ProdutoStoreRequest $request)
30  {
31      try {
32          $novoProduto = Produto::create($request->validated());
33          return (new ProdutoResource($novoProduto))
34              ->additional(["message" => 'Produto criado com sucesso'])
35              ->response()
36              ->setStatusCode(201, "Produto Criado!");
```

Json Resource



Criação de um novo recurso de resposta JSON, específico para o método store.

- Agora, aquelas configurações de resposta, chamadas no objeto **ProdutoResource**, através dos métodos **additional()**, **response()** e **setStatusCode()**, serão configuradas em uma nova classe específica.
- Portanto, devemos gerar um novo recurso com o comando do **artisan**:

```
$> php artisan make:resource ProdutoStoredResource
```

- Vamos modificar o novo recurso para estender a classe **ProdutoResource**, reaproveitando as configurações desta classe.

app > Http > Resources >  ProdutoStoredResource.php > PHP Intelephense >  ProdutoStoredResource

```
3 namespace App\Http\Resources;
4
5 use Illuminate\Http\JsonResponse;
6 use Illuminate\Http\Request;
7
8 class ProdutoStoredResource extends ProdutoResource
9 {
```



```
1  <?php
2
3  namespace App\Http\Resources;
4
5  use Illuminate\Http\JsonResponse;
6  use Illuminate\Http\Request;
7
8  class ProdutoStoredResource extends ProdutoResource
9  {
10
11     public function withResponse(Request $request, JsonResponse $response): void
12     {
13         $response->setStatusCode(201, 'Produto Criado!');
14     }
15
16     public function with(Request $request): array
17     {
18         return [
19             'message' => 'Produto criado com sucesso!!',
20         ];
21     }
22 }
```


Json Resource



Utilização do novo recurso `ProdutoStoredResource` no método `store`.

- As funções `additional()`, `response()` e `setStatusCode()`, foram substituídas, respectivamente pelos métodos `with()` e `withResponse()` da nova classe.
- Portanto, retornamos um novo objeto da classe ***ProdutoStoredResource***, que por sua vez recebe a nova model, criada com dados válidos.

op > Http > Controllers > Api > ProdutoController.php > PHP Intelphense > ProdutoController > store

```
16 class ProdutoController extends Controller
29     public function store(ProdutoStoreRequest $request)
30     {
31         try {
32             return new ProdutoStoredResource(Produto::create($request->validated()));
33         } catch ( ValidationException $error) {
```

- Uma curiosidade, o método `setStatusCode`, é de uma classe do ***Sinfony***, outro *framework* php, o **Laravel** reutiliza vários de seus componentes.
- Documentação sobre Resources de API
- API da Classe `JsonResource`.

Método Store Refatorado

Reduzimos algumas linhas do método store.



- A redução de linhas é evidente, mas além disso o mais importante é a separação de responsabilidades, mantendo um código mais modular e desacoplado possível, trazendo maior legibilidade.
- A separação de responsabilidades é o primeiro princípio S.O.L.I.D. (Single responsibility principle), um conjunto de princípios para aplicações orientadas a objetos que ajuda a construir software de forma flexível, reutilizável e manutenível.

app > Http > Controllers > Api > ProdutoController.php > PHP Inteliphense > ProdutoController > store

```
class ProdutoController extends Controller
{
    public function store(Request $request)
    {
        try {
            $request->validate([
                'nome' => "required | max:20",
                'descricao' => "required | max:300",
                'qtd_estoque' => "required | numeric | min:1",
                'preco' => "required | numeric | min:1.99",
                'importado' => "nullable | boolean",
            ]);
            $produto = $request->all();
            $produto['importado'] = $request->has('importado');
            $novoProduto = Produto::create($produto);
            return (new ProdutoResource($novoProduto))
                ->additional(['message' => 'Produto criado com sucesso!'])
                ->response()
                ->setStatusCode(201, "Produto Criado!");
        } catch (ValidationException $error) {
            throw $error;
        }
    }
}
```

app > Http > Controllers > Api > ProdutoController.php > PHP Inteliphense > ProdutoController > store

```
class ProdutoController extends Controller
{
    public function store(ProdutoStoreRequest $request)
    {
        try {
            return new ProdutoStoredResource(Produto::create($request->validated()));
        } catch (ValidationException $error) {
            throw $error;
        } catch (Exception $error) {
            $message = "Não foi possível criar novo produto!";
            $response = ["Erro:" => $message];
            if (env('APP_DEBUG'))
                $response = [
                    ...$response,
                    'message' => $error->getMessage(),
                    'exception' => $error,
                    'trace' => $error->getTrace(),
                ];
            return response()->json($response, 500);
        }
    }
}
```

Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- Para tratar de erros de forma mais genérica e reaproveitar código, criaremos uma **Superclasse Controller** para a camada da API.
- Portanto, executaremos o comando do *artisan*:

```
$> php artisan make:controller Api/Controller
```

- Implementaremos o método *errorHandler()* que iremos utilizar nas classes filhas, ou seja, todas as nossas classes da camada da API.

app > Http > Controllers > Api > Controller.php > PHP Intelephense > Controller > errorHandler

```
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Exceptions\ExceptionJsonResponse;
6  use Exception;
7
8  class Controller
9  {
10
11      protected function errorHandler(string $message, Exception $error)
12      { ...
17      }
18  }
```

Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- Para criar a classe **ExceptionJsonResponse**, na pasta **app/Exceptions**, executamos o seguinte comando do **artisan**:

```
$>php artisan make:exception ExceptionJsonResponse --render
```

- O parâmetro render irá criar a classe com o método **render()**, o qual utilizaremos para definir como iremos mostrar as informação de erro no formato JSON na resposta HTTP.

```
app > Exceptions > ExceptionJsonResponse.php > PHP Intelephense > ExceptionJsonResponse > render
1  <?php
2
3  namespace App\Exceptions;
4
5  use Exception;
6  use Illuminate\Http\JsonResponse;
7  use Illuminate\Http\Request;
8
9  class ExceptionJsonResponse extends Exception
10 {
11     /**
12      * Render the exception as an HTTP response.
13      */
14     public function render(Request $request): JsonResponse
15 }
```

Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- O método ***errorHandler()*** receberá como argumento a mensagem a ser retornada na requisição e a exceção capturada.
- Implementaremos uma nova classe de exceção para controlar o retorno do erro na requisição, a classe ***ExceptionJsonResponse***.

app > Http > Controllers > Api > Controller.php > ...

```
3 namespace App\Http\Controllers\Api;
4
5 use App\Exceptions\ExceptionJsonResponse;
6 use Exception;
7
8 class Controller
9 {
10
11     protected function errorHandler(string $message, Exception $error, int $statusCode)
12     { ...
18     }
19 }
```


Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- No método **render()** adicionamos a lógica que havíamos criado no segundo **catch** do método **store** do controller **ProdutoController**.
- Esta exceção será usada agora pela superclasse Controller da API, em seu método **errorHandler**.
- As classes filhas, quando precisarem retornar uma exceção, herdarão o método **errorHandler** e poderão passar mensagens de erro e exceções próprias.
- Na linha 17, usamos o

```
app > Exceptions > ExceptionJsonResponse.php > PHP Intelephense > ExceptionJsonResponse
9  class ExceptionJsonResponse extends Exception
13  */
14  public function render(Request $request): JsonResponse
15  {
16      $previous = $this->getPrevious();
17      $statusHttp = $this->getCode() ?? 500;
18      $responseError = [
19          'message' => $this->getMessage(),
20      ];
21
22      if (env('APP_DEBUG'))
23          $responseError = [
24              ...$responseError,
25              'exception' => $previous->getMessage(),
26              'error' => $previous,
27              'trace' => $previous->getTrace()
28          ];
29      return response()
30          ->json($responseError)
31          ->setStatusCode($statusHttp,$this->getMessage());
32  }
33  }
```


Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- O método ***errorHandler()*** da classe pai (superclasse) Controller da nossa camada de api, será responsável por lançar a exceção que controla a resposta HTTP no formata JSON.
- As classes filhas terão acesso ao método e repassarão a mensagem e exceção capturada em seus próprios blocos ***try/catch***.
- Aqui usamos ***named parameters (:message)***, suportado desde a versão 8.0 da linguagem PHP, não sendo necessário passar na mesma ordem em que foram definidos os os parâmetros.

app > Http > Controllers > Api > Controller.php > PHP Intelephense > Controller > errorHandler

```
8  class Controller
10
11  protected function errorHandler(string $message, Exception $error, int $statusCode)
12  {
13      throw new ExceptionJsonResponse(
14          message:$message,
15          previous: $error,
16          code: $statusCode,
17      );
18  }
```

Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- Finalmente alteramos a classe **ProdutoController** da camada da Api, para estender a superclasse (ou classe pai), **Controller**, também no *namespace* da **Api** (**App\Http\Controllers\Api**).
- Com esta modificação, a classe **ProdutoController** tem acesso ao método protegido, *errorHandler()*.

```
✓ PROTOTIPO-20242-LARAVEL-BACKEND-API
└─ app
   └─ Exceptions
      └─ ExceptionJsonResponse.php M
   └─ Http
      └─ Controllers
         └─ Api
            ├── Controller.php M
            └─ ProdutoController.php 2, M
            ├── Controller.php
            ├── FabricanteController.php
            └── FornecedorController.php

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController
15
16 class ProdutoController extends Controller
17 {
18     /**
19      * Display a listing of the
20      */
21     public function index()
22     {
    App\Http\Controllers\Api\Controller
    <?php
    class Controller { }
    <?php
    class Controller
```

Tratamento de Erros

Superclasse Controller da Api para tratamento de Erros



- Atualizamos então o método **store()**, que agora apenas chamará o metodo herdado da classe pai (superclasse) **Controller**, **errorHandler()**.
- Com a refatoração feita no código, reduzimos mais uma vez drasticamente a quantidade de linhas do método **store()** da classe **ProdutoController**.
- Mas além de reduzir linhas, criamos estrutura que serão reaproveitadas nos próximos métodos e em outras classes da nossa camada da API.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store
16  class ProdutoController extends Controller
63      public function store(ProdutoStoreRequest $request)
64  {
65      try {
66          return new ProdutoStoredResource(Produto::create($request->validated()));
67      } catch (Exception $error) {
68          return $this->errorHandler("Erro ao criar novo produto!!", $error);
69      }
70  }
```



Refatoração do método Store

Aplicação do primeiro princípio S. O. L. I. D. da responsabilidade única (*Single Responsibility*)

- Resultado final da refatoração do método **store** e comparação com o código anterior:

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store

```
16 class ProdutoController extends Controller
29     public function store(Request $request)
30     {
31         try {
32             $request->validate([
33                 "nome" => "required | max:20",
34                 "descricao" => "required | max:300",
35                 "qtd_estoque" => "required | numeric | min:1",
36                 "preco" => "required | numeric | min:1.99",
37                 "importado" => "nullable | boolean",
38             ]);
39             $produto = $request->all();
40             $produto['importado'] = $request->has('importado');
41             $novoProduto = Produto::create($produto);
42             return (new ProdutoResource($novoProduto))
43                 ->additional(["message" => 'Produto criado com sucesso!'])
44                 ->response()
45                 ->setStatusCode(201, "Produto Criado!");
46         } catch ( ValidationException $error) {
47             throw $error;
48         } catch (Exception $error) {
49             $message = "Não foi possível criar novo produto!";
50             $response = ["Erro:" => $message];
51             if (env('APP_DEBUG'))
52                 $response = [
53                     ...$response,
54                     'message' => $error->getMessage(),
55                     'exception' => $error,
56                     'trace' => $error->getTrace(),
57                 ];
58             return response()->json($response, 500);
59         }
60     }
```

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > store

```
16 class ProdutoController extends Controller
63     public function store(ProdutoStoreRequest $request)
64     {
65         try {
66             return new ProdutoStoredResource(Produto::create($request->validated()));
67         } catch (Exception $error) {
68             return $this->errorHandler("Erro ao criar novo produto!!", $error);
69         }
70     }
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
```

Método Update

Método update do CRUD utilizando a mesma estrutura do método store



- Para o método update, precisaremos criar as classes **ProdutoUpdateRequest** e **ProdutoUpdatedResource**, nos mesmos passos realizados para o método **store()**.
- Já o controle da mensagem de erros, no bloco **catch**, reaproveitamos o método **errorHandler()**, alterando apenas a mensagem de erro a ser retornada com a exceção.
- Como exercício, crie o método **update** da classe **ProdutoController**,
- **Observação**: na validação da atualização, **não é obrigatório** enviar todos os campos.

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > update

```
16 class ProdutoController extends Controller
83     public function update(ProdutoUpdateRequest $request, Produto $produto)
84     {
85         try {
86             $produto->update($request->validated());
87             return new ProdutoUpdatedResource($produto);
88         } catch (Exception $error) {
89             return $this->errorHandler("Erro ao atualizar produto!!", $error);
90         }
91     }
```


Método Destroy

Método destroy completa as operações de CRUD da API Reste



- O método **destroy()** é mais simples, não sendo necessário criar uma classe de recurso de api apenas para ele.
- Usaremos a classe **ProdutoResource** para retornar o produto removido no formato JSON.
- E apenas o método **additional()** para enviar uma mensagem junto ao produto removido.

app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > destroy

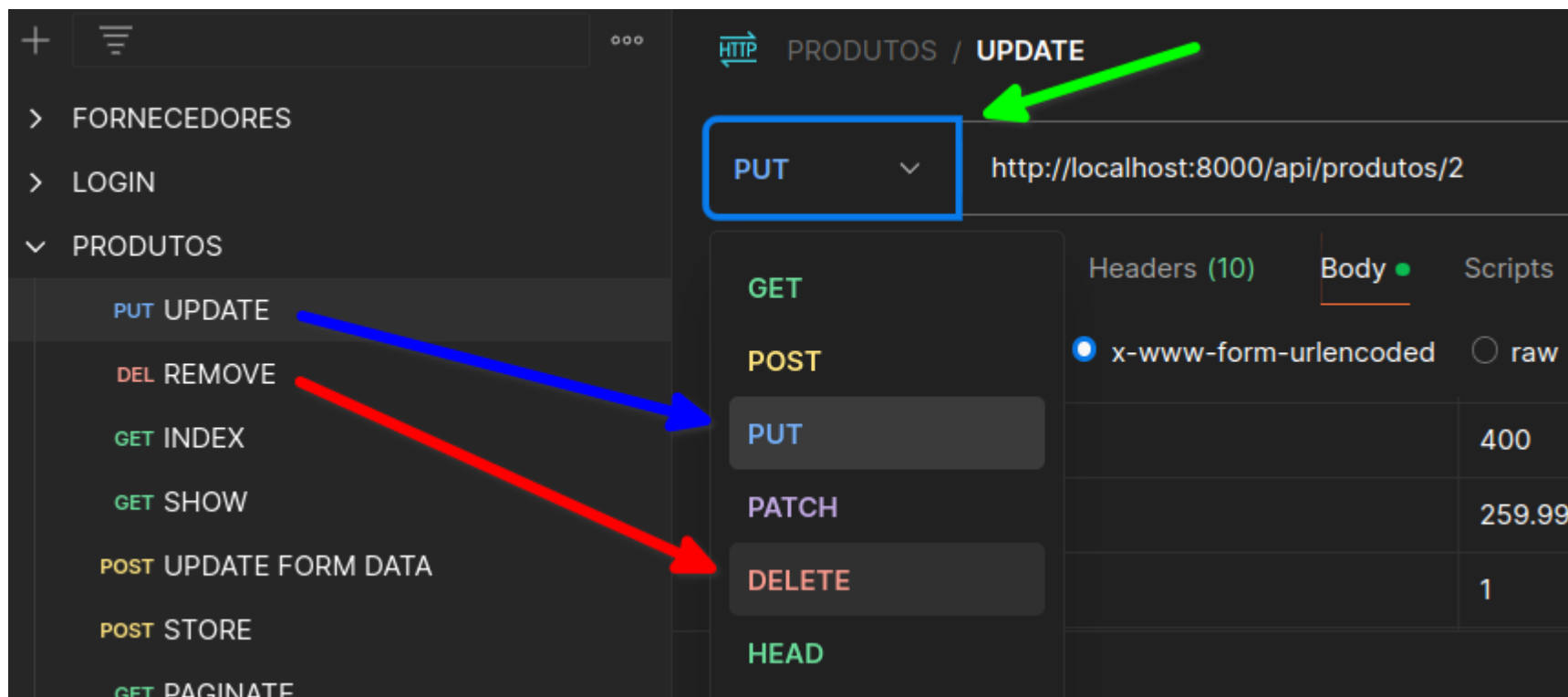
```
16  class ProdutoController extends Controller
96      public function destroy(Produto $produto)
97      {
98
99          try {
100              $produto->delete();
101              return (new ProdutoResource($produto))->additional(["message" => "Produto removido!!!"]);
102          } catch (Exception $error) {
103              return $this->errorHandler("Erro ao remover produto!!", $error);
104          }
105
106      }
107  }
108
```


UPDATE e o DESTROY no Postman

Teste de rotas PUT e DELETE no Cliente HTTP Postman



- Para testar as rotas de update (**PUT**), basta selecionar o verbo correspondente no **select** antes da barra de endereços, salve a requisição como nome **UPDATE** para facilitar a identificação.
- Faça o mesmo procedimento para gerar uma requisição com o verbo **DELETE**, também salve com o nome fácil de identificar a requisição, **REMOVE** ou **DELETE**.





1) Criar a API para três tabelas do seu projeto/TCC:

- Desenvolver os métodos para as rotas de **POST**, **PUT** e **DELETE**.
- Usar classes do tipo **Form Requests** para as validações nos métodos **store** e **update**.
- Usar a injeção de dependência da Model (**Route Model Binding**).
- Utilizar as classes **Resources** específicas para **store** e **update**.
- Envie o link do repositório remoto e privado na atividade referente a esta aula no moodle, ou utilize o mesmo projeto da atividade anterior (**index** e **show**).
- Adicionar o professor como colaborador (**settings/colaborators**)
- Salve na pasta **database/dump** um dump do seu banco de dados com os dados. Mantenha a camada de modelo e **migrations** corretamente configuradas.
- Exporte as coleções do cliente HTTP e salve na pasta **test/api** do seu projeto.



DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009

Fielding, Roy Thomas. ***Architectural Styles and the Design of Network-based Software Architectures***. Doctoral dissertation, University of California, Irvine, 2000.

Status HTTP. Mozilla Developer Network, Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>
<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>, acesso em: 07/11/2024

FIELDING, Roy Thomas, **Architectural Styles and the Design of Network-based Software Architectures**, Dissertation, University of California – Irvine. Disponível em: , Acesso em: 07/11/2024

<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

LARAVEL, **Route Model Binding**. Disponível em: <https://laravel.com/docs/11.x/routing#route-model-binding>, Acesso em: 07/11/2024.

LARAVEL, **Rest Ful Naming Resource Routes**. Disponível em: <https://laravel.com/docs/11.x/controllers#restful-naming-resource-routes>, Acesso em: 07/11/2024.

SOLID no Laravel. Disponível em: <https://dev.to/thiagoluna/solid-no-laravel-aplicando-principios-e-boas-praticas-para-entregar-melhores-solucoes-1ogh>, Acesso em: 07/11/2024.

Princípios de SOLID: O que são e como Aplicá-los no php com Laravel. Disponível em: <https://dev.to/lucascavalcante/principios-solid-o-que-sao-e-como-aplica-los-no-php-laravel-parte-01-responsabilidade-unica-3mjj>, Acesso em: 07/11/2024.

Alura – Introdução ao Postman – Disponível em: <https://www.youtube.com/watch?v=op81bMbgZXs>, Acesso em: 07/11/2024

Canal Dias de Dev – DIAS, Vinicius. Disponível em: <https://www.youtube.com/@DiasDeDev/videos>, Acesso em: 07/11/2024.